Get Notion free

69-1-ITCS258-Project Description

© Faculty of ICT, Mahidol University. All rights reserved.

Faculty of Information and Communication Technology

ITCS258 Backend Application Development

Unit Project: Hotel Booking REST API System

Assigned Date

August 7, 2026

Submission Deadline

November 20, 2026, at 11:55 PM BKK Time (UTC+7)

Note: Late submissions will be accepted.

Project Overview

In this group project, students are required to **design, implement, test, and deploy a production-ready RESTful API** for a **Hotel Booking System**, based on the requirements defined in the **Hotel Booking Specification**.

The project focuses on **back-end system design and engineering**, emphasizing **security, performance, reliability, testing, and deployment**. Students will apply concepts learned throughout the course to build a real-world API that web or mobile front-end applications can consume.

This project is designed to simulate **professional back-end development workflows**, including authentication, role-based access control, caching, testing strategies, containerization, and deployment.

Group Requirements

- Each group consists of **3 students**
- All members must contribute to:
  - Code development
  - Testing
  - Documentation
- Code collaboration must follow academic integrity guidelines

Learning Outcomes

By completing this project, students are expected to:

- Implement a REST API that fulfills all hotel booking functional requirements.
- Satisfy key non-functional requirements, including security, performance, reliability, scalability, maintainability, observability, and deployment constraints.
- Apply concepts in:
  - **Authentication and Authorization** (password hashing, session or token-based auth, secure handling of credentials).
  - **Role-Based Access Control (RBAC)** to enforce business rules for Regular Users and Admins.
  - **REST API caching, rate limiting, and performance tuning**.
  - **API documentation** using Swagger / OpenAPI.
  - **Automated testing** (unit, integration, and end-to-end).
  - **Dockerization** suitable for production-like deployment.
  - **Deployment** to a public server (e.g., cloud VM or platform).

System Scope

From the Hotel Booking Specification, the Hotel Booking REST API must support the following user roles:

- **Guests** (unauthenticated users)

- **Registered User**
- **Admin**

The API must provide endpoints for:

- User authentication and authorization
- Hotel room management
- Room searching and filtering
- Booking creation and management
- Booking status updates and notifications
- System health and operational readiness

All functional and non-functional requirements must comply with the provided Hotel Booking Specification.

Technical Stack (Required)

Students must use the following technologies:

- **Programming Language**: TypeScript
- **Backend Framework**: NestJS
- **Database**: MySQL or any relational database management systems (via ORM such as Prisma or TypeORM)
- **Authentication**: JWT (JSON Web Token)
- **API Documentation**: Swagger / OpenAPI
- **Caching & Rate Limiting**: Redis (or equivalent)
- **Testing**: Jest + Supertest
- **Containerization**: Docker & Docker Compose
- **Deployment**: a provided or public server (e.g., cloud VM or platform).

Project Tasks

Task1: Core REST API Design & Implementation

Students must implement RESTful endpoints that satisfy all **functional requirements** in the Hotel Booking Specification, including but not limited to:

- User Accounts & Roles
  - Sign up, login, logout, view and update profile, supporting Regular User and Admin roles and their capabilities.
- Rooms
  - Admin CRUD for rooms, including image metadata, capacity, price per night, and room state (active/deactivated).
  - Public/authorized endpoints for listing and viewing room details, including associated image data.
- Bookings
  - Logged-in users can create and manage their own bookings with validation of dates and prevention of double booking.
  - Booking statuses: Pending (default), Approved, Cancelled, Paid, with Admin ability to view and update any booking.
- Search & Filters
  - Search rooms by date range and capacity (or similar attributes) and show only available rooms for selected dates.
- Notifications (Simple)
  - Record events for booking creation and cancellation so that a frontend can retrieve them later.
- System Behavior & Operations
  - Clear success and error messages for invalid data, missing resources, and normal operations.
  - Health-check endpoint to verify system status, support for Docker execution, and deployability to a server.

All endpoints must:

- Follow RESTful naming conventions
- Return appropriate HTTP status codes
- Provide clear and consistent error messages

Task 2: Authentication & Authorization

Students must:

- Implement secure user registration and login following NFR-1 and NFR-2, including password hashing and safe credential storage.
- Use a token-based mechanism (e.g., JWT) or equivalent to authorize access to protected endpoints and avoid exposing sensitive data in responses or logs (NFR-4).
- Ensure only authenticated users can create and manage their own bookings and that only Admins can manage rooms and all bookings (NFR-2, NFR-3).

Task 3: Role-Based Access Control (RBAC)

Students must:

- Define at least two roles: Regular User and Admin, as required by FR-5.

- Define at least two roles: Regular User and Admin, as required by FR-3.
- Enforce role checks at the API layer, such that:
  - Regular Users can only manage their own bookings and retrieve their own data (FR-6, FR-23, FR-24).
  - Admins can manage all rooms (FR-8 to FR-16) and all bookings (FR-25, FR-26).

RBAC must be implemented using guards, middleware, or equivalent backend mechanisms.

Task 4: REST API Caching, Rate Limiting & Performance

Students must:

- Design endpoints and queries so that common operations (view rooms, search, view bookings) respond within reasonable time and can handle concurrent users (NFR-5, NFR-7).
- Apply at least:
  - One **caching strategy** for frequently accessed resources (such as room list or room details), while respecting data freshness requirements.
  - One **rate-limiting** strategy on sensitive or high-traffic endpoints (authentication, booking creation) to improve robustness.
- Document the chosen performance, caching, and rate-limiting strategies in the project README.

Students should demonstrate:

- Where caching is used
- Why caching is applied to those endpoints
- How rate limiting improves system stability

Task 5: API Documentation (Swagger)

Students must:

- Provide interactive API documentation using Swagger UI or an equivalent OpenAPI-based tool, covering:
  - All main endpoints for users, rooms, bookings, search, notifications, and health check, including request/response schemas and HTTP status codes.
  - Authentication requirements and example tokens or flows.
- Ensure the documentation endpoint is accessible and up to date with the implemented API.

Task 6: API Testing

Students must write an API Testing document and implement automated tests for maintainability and reliability, aligned with NFR-13.

- Unit Testing
  - Cover core services and business logic (e.g., booking validation, date conflict checks, role checks, and status transitions).
- Integration Testing
  - Test API endpoints (Controllers) with a test database, including:
    - User registration/login.
    - Room CRUD operations as Admin.
    - Booking creation and conflict prevention, status updates, and search behavior.
- End-to-End (E2E) Testing
  - Simulate realistic user flows (e.g., user sign-up → login → search available rooms → create booking → cancel booking), validating responses and resulting database state.

Each test category must include:

- Clear test cases
- Meaningful assertions
- Automated execution via test commands
- Tests should clean up after completion

Students should provide scripts (e.g., `npm test`) and clear instructions in the README on how to run all tests.

Task 7: Dockerization for Production

Students must:

- Containerize the back-end service to satisfy FR-35 and NFR-18, using a `Dockerfile` optimized for production (multi-stage build is recommended but not required).
- Externalize configuration (database URL, secrets, ports) using environment variables, consistent with NFR-19.
- Provide a `docker-compose.yml` or equivalent, if needed, to run the API with its database on a developer machine.

Task 8: Deployment

Students must:

- Deploy the containerized back-end to a server environment aligned with FR-38 and NFR-20 (e.g., school-provided VM, cloud provider, or container platform).
- Ensure:
  - The Dockerized service can run reliably on the target server and connect to the configured database.
  - Health-check and logging are usable for monitoring (NFR-15 to NFR-17).

- Provide:
  - URL of the deployed API.
  - Any credentials for test accounts and roles (in a secure, instruction-compliant way).
  - Step-by-step deployment description in the README.

Documentation & Submission Requirements

Students must submit the following on Classroom50 [Link]:

- Classroom50
  - For the first time, when you access Project on Classroom50, the student who has the smallest ID needs to create a team. The team's name needs to be in the format of **691_ITCS258_GroupY**, where Y is the team number. For other team members, you need to select the correct team on Classroom50.
- Source code
  - REST API source code (excluding `node_modules` or similar vendor directories).
  - Configuration templates or `.env.example` files, but not real secrets.
  - Well-structured, modular code
  - Clear comments and naming conventions
- Tests
  - All unit, integration, and E2E test code with clear instructions to run them.
- API Documentation
  - Swagger/OpenAPI specification file(s) and instructions to access the Swagger UI endpoint.
- Docker & Deployment Files
  - `Dockerfile`, optional `docker-compose.yml`, and deployment scripts or instructions.
- README.md
  - Project overview and architecture.
  - Environment setup instructions
  - How to run in development (with and without Docker).
  - API usage examples
  - API Testing document, how to run tests, and test results.
  - How to build and run Docker image(s).
  - How the system is deployed (environment, commands, architecture diagram if available).

Grading process

From this project onward, grading will include **peer code review**:

- Each team will submit their own implementation (**Dev Group**).
- Each Dev Group will be assigned another team as a **Review Group** to review the code and score the Dev Group. The output is the grading report.
- If the Review Group finds genuine issues or mistakes, the Review Group may receive **bonus points** (up to 3%).
- The instructors will re-check the review grading. If the Review Group's evaluation is found to be inaccurate, **points will be deducted from the Review Group's project score**.
- Review assignments will be **anonymous**.
- For the review process, submissions will be provided as a **ZIP file**.
- The Review Group need to return review and score by November 24, 2026, at 11:55 PM BKK Time (UTC+7). Otherwise, the Review Group will get ZERO for your project.

Academic Integrity

Please do your own group work. Your survival in the subsequent courses heavily depends on the programming skills that you acquire in this course. Though students are allowed and encouraged to discuss ideas with others and search for online sources, the actual solutions must be written by themselves without being dictated or looking at others' code. Collaboration in writing solutions with other groups is not allowed, as it would be unfair to other students. It is better to submit a broken program that is a result of your own effort than take somebody else's work for your own credit! Students who know how to obtain the solutions are encouraged to help others by guiding them and teaching them the core material needed to complete the project, rather than giving away the solutions.

Unit Project Marking Criteria

Hotel Booking REST API System (Total: 100 points)

> Important Note
>
> All Functional Requirements (FR-1 to FR-38) defined in the *Hotel Booking Specification* must be implemented.
>
> Missing or partially implemented FRs will result in **direct score deduction** across relevant sections.

Core REST API & Functional Requirements (30 pts)

All core REST endpoints must satisfy the **Hotel Booking Specification FRs**.

Criteria	Level 2	Level 1	Level 0
1. User Accounts & Roles (FR-1–FR-7)	All required user endpoints (sign up, login/logout, view/update profile) are implemented and work correctly for both Regular User and Admin roles.	Some user endpoints are missing or partially working, or behavior does not fully match the specification.	No meaningful user account API or roles implemented.
2. Rooms API (FR-8–FR-16)	Room CRUD for Admin and room listing/detail endpoints are fully implemented and behave correctly, including image metadata handling.	Some room operations are missing or incorrect (e.g., cannot delete/deactivate, image references incomplete).	No room management API implemented.
3. Bookings API (FR-17–FR-26)	Booking creation, listing, detail, and Admin status updates are implemented with correct validation and status flow.	Booking API exists but has flaws (e.g., missing some views, status changes limited, or incorrect responses).	No booking API implemented.
4. Date Validation & No Double Booking (FR-19–FR-20)	System strictly validates date ranges and reliably prevents double bookings for a room.	Date checks or conflict checks are partially implemented or unreliable under some conditions.	No validation or conflict handling.
5. Search & Filters (FR-27–FR-29)	Room search by date range and capacity works correctly and returns only truly available rooms.	Search endpoints exist but are incomplete or sometimes return unavailable rooms.	No search/filter functionality.
6. Notifications (FR-30–FR-31)	Booking creation and cancellation events are recorded and accessible via API for frontend use.	Events are recorded in a limited or inconsistent way, or only partially exposed.	No notification/event recording.
7. System Messages & Error Handling (FR-32–FR-34)	API consistently returns clear success and error messages with appropriate HTTP status codes.	Messages exist but are sometimes unclear, inconsistent, or use poor status codes.	No meaningful error or success messages.

Security, Authentication & RBAC (15 pts)

Criteria	Level 2	Level 1	Level 0
8. Authentication Implementation (NFR-1–NFR-2)	Secure registration and login are implemented (password hashing, token/session), and only logged-in users can manage bookings.	Authentication exists but has weaknesses (e.g., weak hashing, missing protections on some booking endpoints).	No authentication, or passwords stored in plain text.
9. Role-Based Access Control (NFR-3, FR-5–FR-7, FR-25–FR-26)	RBAC correctly restricts room CRUD and all bookings operations to Admins; users can manage only their own bookings.	RBAC rules exist but are incomplete (e.g., some Admin-only actions are open, or user data exposure).	No meaningful RBAC; all users can perform all actions.
10. Sensitive Data Protection & Logging (NFR-4, NFR-15–NFR-16)	Sensitive fields are excluded from responses/logs and logs include useful but non-sensitive information.	Some sensitive data occasionally appears in responses or logs, or logging is minimal.	Sensitive data is frequently exposed in responses or logs.

Performance, Caching & Rate Limiting (10 pts)

Criteria	Level 2	Level 1	Level 0
11. Performance-Aware Design (NFR-5–NFR-7, NFR-10–NFR-11)	Endpoints are reasonably fast under normal load; queries and data access patterns consider concurrency and future scaling.	Performance is acceptable but with some obvious inefficiencies; may degrade with concurrent users.	No consideration for performance; frequent timeouts or obvious blocking issues.
12. Caching Strategy	A sensible caching mechanism is implemented and documented for frequently accessed resources without breaking correctness.	Caching is implemented in a limited or ad-hoc way, or poorly documented.	No caching strategy.
13. Rate Limiting & Abuse Protection	Rate limiting or similar controls are correctly applied to sensitive/high-traffic endpoints and documented.	Some basic rate limiting exists but is misconfigured or incomplete.	No rate limiting or abuse protection.

API Documentation (Swagger) (10 pts)

API Documentation (Swagger) (10 pts)

Criteria	Level 2	Level 1	Level 0
14. Swagger/OpenAPI Coverage (FR-37)	Swagger/OpenAPI documentation exists for all core endpoints (users, rooms, bookings, search, notifications, health) with accurate schemas and responses.	Documentation exists but some endpoints, fields, or responses are missing or inaccurate.	No meaningful Swagger/OpenAPI documentation.
15. Usability of API Docs	Swagger UI (or similar) is accessible, easy to navigate, and reflects the current implementation, including authentication instructions.	UI is accessible but hard to use, outdated, or lacks authentication examples.	Docs are inaccessible or non-functional.

API Testing (15 pts)

Students must implement **all three testing levels**.

Criteria	Level 2	Level 1	Level 0
16. Unit Tests (NFR-13)	Core business logic (e.g., date validation, double-booking prevention, role checks) is covered by well-structured unit tests.	Some unit tests exist but cover only a small subset of core logic.	No unit tests.
17. Integration Tests	API endpoints are tested with a test database, including key flows (user auth, room CRUD, booking CRUD, search).	Integration tests exist but are partial or not reliably runnable.	No integration tests.
18. E2E Tests	End-to-end tests simulate realistic user/admin flows and verify both responses and database state.	Some E2E tests exist but are incomplete or flaky.	No E2E tests.
19. Test Execution & Automation	Tests can be run with a simple command (e.g., `npm test`) and are documented in README.	Tests require manual configuration or multiple steps not clearly described.	No clear way to run tests.

Dockerization & Deployment (10 pts)

Criteria	Level 2	Level 1	Level 0
20. Dockerization (FR-35, NFR-18–NFR-19)	A working production-style `Dockerfile` and (optionally) `docker-compose` allow the service to run with externalized configuration.	Dockerization exists but is not optimized, or configuration is partially hard-coded.	No working Docker setup.
21. Health Check & Operations (FR-36)	Health-check endpoint works correctly and is suitable for monitoring and deployment checks.	Health-check exists but returns limited or incorrect information.	No health-check endpoint.
22. Deployment (FR-38, NFR-20)	Application is successfully deployed to a server; README includes URL (if allowed) and clear deployment steps.	Deployment is attempted and partially working or poorly documented.	No deployment or deployment cannot be verified.

Code Quality, Observability & README (10pts)

Criteria	Level 2	Level 1	Level 0
23. Code Organization & Style (NFR-12, NFR-14)	Codebase is clearly structured (layers/modules), consistently formatted, and easy to understand and extend.	Structure and style are mixed; understandable but with duplication or inconsistent naming.	Code is disorganized and very hard to maintain.
24. Logging & Monitoring Hooks (NFR-15–NFR-17)	Meaningful logs for important events and errors are implemented and ready for integration with a monitoring tool.	Some logging exists but is sparse, noisy, or not clearly structured.	No useful logging.
25. README & Project Documentation	README clearly explains setup, configuration, running, testing	README is present but incomplete or missing important steps or clarifications	No README or README not helpful.

	running, testing, Docker, and deployment; mentions limitations and known issues.	steps or clarifications.	

Important Grading Notes

- **Any missing Functional Requirement (FR)** will reduce scores in **multiple sections**
- APIs must **work correctly**, not just compile
- Security violations (e.g., exposed passwords) may cause **severe penalties**
- A partially working system with correct design is better than a copied system